

ALGORITHMS AND DATA STRUCTURES

Practice Exam Paper (2 hours)

Module(s): 25COP923

Solutions

Total Marks: 100

Question 1

[10 marks]

Definitions

Provide informal definitions of the concepts below.

- (a) What is a hash table? Explain how it achieves fast lookup on average. [5]

Solution:

A hash table maps keys to values using a hash function that computes an index into an array of buckets. On average, lookup, insertion and deletion run in $O(1)$ time because the hash function spreads keys roughly uniformly across buckets. Collisions (two keys hashing to the same bucket) are handled via chaining or open addressing.

- (b) What is a greedy algorithm? State one property that must hold for a greedy approach to guarantee an optimal solution. [5]

Solution:

A greedy algorithm builds a solution step by step, always choosing the option that looks best by some local criterion, without reconsidering earlier choices. A greedy approach guarantees optimality when the problem has the greedy-choice property (a locally optimal choice leads to a globally optimal solution) together with optimal substructure (an optimal solution contains optimal solutions to its subproblems).

Question 2

[15 marks]

Algorithm Analysis

What is the worst-case time complexity of the following algorithm in big-O notation? What task is performed by the algorithm? How is that task achieved? (Answer each in the corresponding sub-question.)

Procedure(A, n, target)

1 low $\leftarrow$ 1, high $\leftarrow$ n

2 while low $\leq$ high do

3 mid $\leftarrow$ floor((low + high) / 2)

```

4 if A[mid] = target then
5 return mid
6 else if A[mid] < target then
7 low <- mid + 1
8 else
9 high <- mid - 1
10 return -1

```

(a) Complexity? [5]

Solution:

$O(\log n)$

(b) Task performed? [5]

Solution:

Binary search: find the index of 'target' within a sorted array A, returning -1 if it isn't present.

(c) How achieved? [5]

Solution:

The algorithm repeatedly halves the search range: it compares the middle element to the target and discards the half of the range that cannot contain it, continuing until the target is found or the range is empty. Because the search space halves each iteration, at most $O(\log n)$ comparisons are needed.

Question 3 [20 marks]

Data Structures

Consider a queue (FIFO) implemented as a singly linked list with head (front) and tail (back) pointers, where enqueue adds to the tail and dequeue removes from the head.

(a) Starting from an empty queue, perform the operations: enqueue(3), enqueue(7), enqueue(1), dequeue(), enqueue(9), dequeue(). State the final contents of the queue (front to back) and the value returned by each dequeue call. [8]

Solution:

After enqueue(3), enqueue(7), enqueue(1): [3,7,1]. The first dequeue() removes and returns 3, leaving [7,1]. enqueue(9) gives [7,1,9]. The second dequeue() removes and returns 7, leaving [1,9]. Final queue (front to back): [1, 9]. The dequeue calls returned 3, then 7.

- (b) Suppose each node has fields ‘value’ and ‘next’, and the queue object Q tracks ‘head’ and ‘tail’. Write pseudocode for Enqueue(Q, value). [6]

Solution:

```
Enqueue(Q, value)
1 node <- new Node(value, next = NIL)
2 if Q.head = NIL then
3   Q.head <- node
4   Q.tail <- node
5 else
6   Q.tail.next <- node
7   Q.tail <- node
```

- (c) Write pseudocode for Dequeue(Q), which removes and returns the value at the front of the queue. [6]

Solution:

```
Dequeue(Q)
1 if Q.head = NIL then
2   error "queue is empty"
3 value <- Q.head.value
4 Q.head <- Q.head.next
5 if Q.head = NIL then
6   Q.tail <- NIL
7 return value
```

Question 4

[35 marks]

Graphs

An emergency response coordinator manages a network of towns connected by roads. Each road has an associated travel time. In the event of a flood, the coordinator needs to know the minimum time required to send aid from the central depot town to every other town in the network.

- (a) Explain how this problem can be modelled as a graph problem. [10]

Solution:

Model the road network as a weighted graph: each town is a vertex, each road is an edge (undirected, since roads can be travelled in either direction), and each edge’s weight is the travel time along that road. Finding the minimum time to reach every town from the depot is equivalent to computing single-source shortest-path distances from the depot vertex to every other vertex.

- (b) Design an algorithm that computes these minimum travel times. Describe it in words, give high-level pseudocode, and state which known algorithm it is based on. [15]

Solution:

We use Dijkstra's algorithm for single-source shortest paths with non-negative edge weights.

```

ShortestTimes( $G = (V, E)$ , depot)
1 for each  $v$  in  $V$  do  $\text{dist}[v] \leftarrow \text{infinity}$ 
2  $\text{dist}[\text{depot}] \leftarrow 0$ 
3  $\text{PQ} \leftarrow$  priority queue containing  $(\text{depot}, 0)$ 
4 while  $\text{PQ}$  not empty do
5  $u \leftarrow \text{PQ.extractMin}()$ 
6 for each neighbour  $w$  of  $u$  with edge weight  $t$  do
7 if  $\text{dist}[u] + t < \text{dist}[w]$  then
8  $\text{dist}[w] \leftarrow \text{dist}[u] + t$ 
9  $\text{PQ.insertOrDecreaseKey}(w, \text{dist}[w])$ 
10 return  $\text{dist}$ 

```

- (c) Discuss a suitable representation of the graph and an implementation of your algorithm. Mention any auxiliary data structures and estimate the running time. [10]

Solution:

An adjacency list (a list of outgoing edges and weights per vertex) is efficient here, together with a $\text{dist}[]$ array and a min-heap priority queue for selecting the next closest vertex. With an adjacency list and a binary heap, Dijkstra's algorithm runs in $O((|V| + |E|) \log |V|)$ time.

Question 5

[20 marks]

Minimum Spanning Trees

Let $G = (V, E)$ be a connected, weighted, undirected graph.

- (a) Define what a spanning tree of G is, and what a minimum spanning tree (MST) of G is. [5]

Solution:

A spanning tree of G is a subgraph that contains all of G 's vertices, is connected, and contains no cycles (a tree). A minimum spanning tree is a spanning tree whose total edge weight is the smallest among all possible spanning trees of G .

- (b) State the problem solved by Kruskal's algorithm, giving its input and output. [5]

Solution:

Kruskal's algorithm solves the minimum spanning tree problem for a connected, weighted, undirected graph. Input: a connected undirected graph $G = (V, E)$ with edge weights. Output: a minimum spanning tree of G .

- (c) Run the simple (non-optimised) version of Kruskal's algorithm on the graph with vertices $\{P, Q, R, S, T\}$ and edges P-Q (3), P-R (1), Q-R (4), Q-S (6), R-S (2), R-T (5), S-T (3). List the edges in sorted order, state for each whether it is taken, and give the total weight of the resulting minimum spanning tree. [10]

Solution:

Sorted edges: P-R (1), R-S (2), P-Q (3), S-T (3), Q-R (4), R-T (5), Q-S (6).

P-R (1): no cycle, taken. MST = {P-R}

R-S (2): no cycle, taken. MST = {P-R, R-S}

P-Q (3): no cycle, taken. MST = {P-R, R-S, P-Q}

S-T (3): no cycle, taken. MST = {P-R, R-S, P-Q, S-T}

Q-R (4): would create a cycle (P-Q-R-P), rejected.

R-T (5): R and T are already connected via R-S-T, rejected.

Q-S (6): already connected, rejected.

5 vertices need 4 edges for a spanning tree, already reached after S-T, so all remaining edges are rejected. Total MST weight = $1 + 2 + 3 + 3 = 9$.